

1. Create the report script

Create:

```
mkdir -p /opt/mysql-dba-report
```

```
cd /opt/mysql-dba-report
```

```
vi mysql_daily_report.py
```

```
#!/usr/bin/env python3
```

```
import os
```

```
import socket
```

```
import shutil
```

```
import subprocess
```

```
from datetime import datetime
```

```
import mysql.connector
```

```
import psutil
```

```
# =====
```

```
# CONFIGURATION
```

```
# =====
```

```
MYSQL_HOST = "127.0.0.1"
```

```
MYSQL_PORT = 3306
```

```
MYSQL_USER = "dba_report"
```

```
MYSQL_PASSWORD = "CHANGE_ME"
```

```
REPORT_DIR = "/opt/mysql-dba-report/reports"
```

```
# MySQL data directory
```

```
MYSQL_DATA_DIR = "/var/lib/mysql"
```

```
# Warning thresholds
```

```
DISK_WARNING = 80
```

```
DISK_CRITICAL = 90
```

```
MEMORY_WARNING = 80
```

```
MEMORY_CRITICAL = 90
```

```
CONNECTION_WARNING = 80
```

```
CONNECTION_CRITICAL = 90
```

```
LONG_QUERY_SECONDS = 300
```

```
TOP_QUERIES = 5
```

```
ERROR_LOG_LIMIT = 10
```

```
# =====
```

```
# MYSQL CONNECTION
```

```
# =====
```

```
def mysql_connection():
```

```

return mysql.connector.connect(
    host=MYSQL_HOST,
    port=MYSQL_PORT,
    user=MYSQL_USER,
    password=MYSQL_PASSWORD
)

# =====
# HTML HELPERS
# =====

def status(value):
    return value

def badge(text, level):
    colors = {
        "GREEN": "#198754",
        "AMBER": "#ffc107",
        "RED": "#dc3545",
        "INFO": "#0d6efd"
    }

    color = colors.get(level, "#6c757d")

    return f"""
    <span style="
      background:{color};
      color:{'#000' if level == 'AMBER' else '#fff'};
      padding:4px 9px;
      border-radius:12px;
      font-weight:bold;
      font-size:12px;">
      {text}
    </span>
    """

def table(headers, rows):
    html = "<table><tr>"

    for h in headers:
        html += f"<th>{h}</th>"

    html += "</tr>"

    for row in rows:
        html += "<tr>"

        for value in row:
            html += f"<td>{value}</td>"

        html += "</tr>"

    html += "</table>"

```

```

return html

def escape(value):
    if value is None:
        return ""

    return (
        str(value)
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
    )

# =====
# SERVER HEALTH
# =====

def get_server_health():

    cpu = psutil.cpu_percent(interval=1)

    memory = psutil.virtual_memory()

    disk = shutil.disk_usage("/")

    disk_total = disk.total / 1024**3
    disk_used = disk.used / 1024**3
    disk_free = disk.free / 1024**3

    disk_percent = (disk.used / disk.total) * 100

    return {
        "cpu": cpu,
        "memory_total": memory.total / 1024**3,
        "memory_used": memory.used / 1024**3,
        "memory_free": memory.available / 1024**3,
        "memory_percent": memory.percent,
        "disk_total": disk_total,
        "disk_used": disk_used,
        "disk_free": disk_free,
        "disk_percent": disk_percent
    }

# =====
# MYSQL DATA DIRECTORY SIZE
# =====

def directory_size(path):

    total = 0

    if not os.path.exists(path):
        return 0

```

```

for root, dirs, files in os.walk(path):

    for file in files:

        try:
            total += os.path.getsize(
                os.path.join(root, file)
            )
        except (PermissionError, FileNotFoundError):
            pass

    return total / 1024**3

# =====
# MYSQL INFORMATION
# =====

def mysql_info(conn):

    cursor = conn.cursor(dictionary=True)

    cursor.execute("""
        SELECT
            VERSION() AS version,
            @@hostname AS hostname,
            @@port AS port,
            @@datadir AS datadir,
            @@log_bin AS log_bin,
            @@binlog_format AS binlog_format
    """)

    return cursor.fetchone()

# =====
# UPTIME
# =====

def mysql_uptime(conn):

    cursor = conn.cursor(dictionary=True)

    cursor.execute("""
        SHOW GLOBAL STATUS LIKE 'Uptime'
    """)

    row = cursor.fetchone()

    seconds = int(row["Value"])

    days = seconds // 86400
    hours = (seconds % 86400) // 3600
    minutes = (seconds % 3600) // 60

    return f"{days}d {hours}h {minutes}m"

```

```
# =====  
# DATABASE SIZE  
# =====
```

```
def database_sizes(conn):
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("""  
        SELECT  
            table_schema AS database_name,  
            ROUND(  
                SUM(data_length) / 1024 / 1024 / 1024,  
                2  
            ) AS data_gb,  
  
            ROUND(  
                SUM(index_length) / 1024 / 1024 / 1024,  
                2  
            ) AS index_gb,  
  
            ROUND(  
                (  
                    SUM(data_length)  
                    + SUM(index_length)  
                ) / 1024 / 1024 / 1024,  
                2  
            ) AS total_gb  
  
        FROM information_schema.tables  
  
        WHERE table_schema NOT IN  
        (  
            'information_schema',  
            'performance_schema',  
            'mysql',  
            'sys'  
        )  
  
        GROUP BY table_schema  
  
        ORDER BY total_gb DESC  
    """)
```

```
    return cursor.fetchall()
```

```
# =====  
# CONNECTIONS  
# =====
```

```
def connection_stats(conn):
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("""
```

```

SELECT
    COUNT(*) AS total,
    SUM(COMMAND <> 'Sleep') AS active,
    SUM(COMMAND = 'Sleep') AS sleeping
FROM information_schema.processlist
"""
)

```

```
stats = cursor.fetchone()
```

```

cursor.execute("""
    SHOW VARIABLES LIKE 'max_connections'
""")

```

```
max_conn = int(cursor.fetchone()["Value"])
```

```
stats["max_connections"] = max_conn
```

```

stats["utilization"] = (
    stats["total"] / max_conn * 100
)

```

```
return stats
```

```

# =====
# TOP RUNNING QUERIES
# =====

```

```
def running_queries(conn):
```

```
    cursor = conn.cursor(dictionary=True)
```

```

    cursor.execute("""
        SELECT
            ID,
            USER,
            HOST,
            DB,
            COMMAND,
            TIME,
            STATE,
            LEFT(INFO, 500) AS QUERY_TEXT
    """

```

```
        FROM information_schema.processlist
```

```
        WHERE COMMAND <> 'Sleep'
```

```
        ORDER BY TIME DESC
```

```

        LIMIT %s
    """, (TOP_QUERIES,))

```

```
    return cursor.fetchall()
```

```

# =====
# LONG RUNNING QUERIES

```

```

# =====

def long_queries(conn):

    cursor = conn.cursor(dictionary=True)

    cursor.execute("""
        SELECT
            ID,
            USER,
            HOST,
            DB,
            COMMAND,
            TIME,
            STATE,
            LEFT(INFO, 500) AS QUERY_TEXT

        FROM information_schema.processlist

        WHERE COMMAND <> 'Sleep'
        AND TIME >= %s

        ORDER BY TIME DESC
        """, (LONG_QUERY_SECONDS,))

    return cursor.fetchall()

# =====
# ERROR LOG
# =====

def error_log(conn):

    cursor = conn.cursor(dictionary=True)

    try:

        cursor.execute("""
            SELECT
                LOGGED,
                PRIO,
                ERROR_CODE,
                SUBSYSTEM,
                LEFT(DATA, 500) AS MESSAGE

            FROM performance_schema.error_log

            WHERE PRIO IN ('ERROR', 'Warning')

            ORDER BY LOGGED DESC

            LIMIT %s
            """, (ERROR_LOG_LIMIT,))

        return cursor.fetchall()

```

```

except Exception:

    return []

# =====
# REPLICATION
# =====

def replication_status(conn):

    cursor = conn.cursor(dictionary=True)

    try:

        cursor.execute("SHOW REPLICA STATUS")

        row = cursor.fetchone()

        if not row:
            return None

        return {
            "io": row.get("Replica_IO_Running"),
            "sql": row.get("Replica_SQL_Running"),
            "lag": row.get("Seconds_Behind_Source"),
            "last_io_error": row.get("Last_IO_Error"),
            "last_sql_error": row.get("Last_SQL_Error")
        }

    except Exception as e:

        return {
            "error": str(e)
        }

# =====
# BINARY LOG
# =====

def binary_logs(conn):

    cursor = conn.cursor(dictionary=True)

    try:

        cursor.execute("SHOW BINARY LOGS")

        rows = cursor.fetchall()

        total_size = sum(
            int(row["File_size"])
            for row in rows
        )

        return {

```



```

        "files": len(rows),
        "size_gb": total_size / 1024**3
    }

```

except Exception:

```

    return None

```

```

# =====
# MYSQL STATUS
# =====

```

def mysql_status(conn):

```

    cursor = conn.cursor(dictionary=True)

```

```

    variables = [
        "Threads_connected",
        "Threads_running",
        "Threads_created",
        "Aborted_clients",
        "Aborted_connects",
        "Created_tmp_tables",
        "Created_tmp_disk_tables"
    ]

```

```

    placeholders = ",".join(["%s"] * len(variables))

```

```

    cursor.execute(
        f"""
        SHOW GLOBAL STATUS
        WHERE Variable_name IN ({placeholders})
        """,
        variables
    )

```

```

    result = {}

```

```

    for row in cursor.fetchall():
        result[row["Variable_name"]] = row["Value"]

```

```

    return result

```

```

# =====
# REPORT GENERATION
# =====

```

def generate_report():

```

    os.makedirs(REPORT_DIR, exist_ok=True)

```

```

    conn = mysql_connection()

```

```

    info = mysql_info(conn)
    uptime = mysql_uptime(conn)

```

```

health = get_server_health()

db_sizes = database_sizes(conn)

connections = connection_stats(conn)

running = running_queries(conn)

long_running = long_queries(conn)

errors = error_log(conn)

replication = replication_status(conn)

binlogs = binary_logs(conn)

mysqlstats = mysql_status(conn)

mysql_data_size = directory_size(
    info["datadir"]
)

hostname = info["hostname"]

now = datetime.now()

# -----
# OVERALL STATUS
# -----

overall = "GREEN"

if health["disk_percent"] >= DISK_CRITICAL:
    overall = "RED"

elif health["disk_percent"] >= DISK_WARNING:
    overall = "AMBER"

if health["memory_percent"] >= MEMORY_CRITICAL:
    overall = "RED"

elif health["memory_percent"] >= MEMORY_WARNING:
    if overall != "RED":
        overall = "AMBER"

if connections["utilization"] >= CONNECTION_CRITICAL:
    overall = "RED"

elif connections["utilization"] >= CONNECTION_WARNING:
    if overall != "RED":
        overall = "AMBER"

if replication and replication.get("io") == "No":
    overall = "RED"

if replication and replication.get("sql") == "No":

```

```
        overall = "RED"

# -----
# HTML
# -----

html = f"""
<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>MySQL Daily DBA Health Report</title>

<style>

body {{
    font-family: Arial, sans-serif;
    background:#f5f7fa;
    margin:30px;
    color:#222;
}}

h1 {{
    color:#17365d;
}}

h2 {{
    background:#17365d;
    color:white;
    padding:10px;
    border-radius:4px;
}}

table {{
    width:100%;
    border-collapse:collapse;
    margin-bottom:25px;
    background:white;
}}

th {{
    background:#d9eaf7;
    padding:8px;
    border:1px solid #ccc;
}}

td {{
    padding:8px;
    border:1px solid #ddd;
}}

.card {{
    background:white;
```

```

padding:20px;
margin-bottom:20px;
border-radius:6px;
}}

.warning {{
  color:#9a6700;
  font-weight:bold;
}}

.critical {{
  color:#c00;
  font-weight:bold;
}}

</style>

</head>

<body>

<h1>MySQL Daily DBA Health Report</h1>

<div class="card">

<b>Date:</b> {now.strftime('%d-%b-%Y %H:%M:%S')}<br>
<b>Server:</b> {hostname}<br>
<b>MySQL Version:</b> {info['version']}<br>
<b>Overall Status:</b> {badge(overall, overall)}

</div>

<h2>1. Server Health</h2>

{table(
  ["Metric", "Value", "Status"],
  [
    ["CPU", f"{health['cpu']:.1f}%", badge(
      "WARNING" if health["cpu"] >= 80 else "OK",
      "AMBER" if health["cpu"] >= 80 else "GREEN"
    )],

    ["Memory", f"{health['memory_percent']:.1f}%",
    badge(
      "WARNING" if health["memory_percent"] >= MEMORY_WARNING else "OK",
      "RED" if health["memory_percent"] >= MEMORY_CRITICAL
      else "AMBER" if health["memory_percent"] >= MEMORY_WARNING
      else "GREEN"
    )],

    ["Disk", f"{health['disk_percent']:.1f}%",
    badge(
      "CRITICAL" if health["disk_percent"] >= DISK_CRITICAL
      else "WARNING" if health["disk_percent"] >= DISK_WARNING
      else "OK",

```

```

        "RED" if health["disk_percent"] >= DISK_CRITICAL
        else "AMBER" if health["disk_percent"] >= DISK_WARNING
        else "GREEN"
    )],

    ["Disk Total", f"{health['disk_total']:.2f} GB", ""],
    ["Disk Used", f"{health['disk_used']:.2f} GB", ""],
    ["Disk Free", f"{health['disk_free']:.2f} GB", ""],
    ["MySQL Data Directory", f"{mysql_data_size:.2f} GB", ""],
    ["MySQL Uptime", uptime, ""],
]
}}

```

<h2>2. Database Size</h2>

```

{table(
  ["Database", "Data GB", "Index GB", "Total GB"],
  [
    [
      escape(r["database_name"]),
      r["data_gb"],
      r["index_gb"],
      r["total_gb"]
    ]
    for r in db_sizes
  ]
)}

```

<h2>3. Connections</h2>

```

{table(
  ["Metric", "Value"],
  [
    ["Maximum Connections", connections["max_connections"]],
    ["Current Connections", connections["total"]],
    ["Active Connections", connections["active"]],
    ["Sleeping Connections", connections["sleeping"]],
    ["Connection Utilization",
      f"{connections['utilization']:.1f}%"]
  ]
)}

```

<h2>4. Top 5 Currently Running Queries</h2>

```

{table(
  ["ID", "User", "DB", "Runtime", "State", "Query"],
  [
    [
      r["ID"],
      escape(r["USER"]),
      escape(r["DB"]),
      f"{r['TIME']} sec",
      escape(r["STATE"]),
      escape(r["QUERY_TEXT"])
    ]
  ]
)}

```

```

    ]
    for r in running
  ]
}}

```

<h2>5. Long Running Queries</h2>

```

{table(
  ["ID", "User", "DB", "Runtime", "State", "Query"],
  [
    [
      r["ID"],
      escape(r["USER"]),
      escape(r["DB"]),
      f"{r['TIME']} sec",
      escape(r["STATE"]),
      escape(r["QUERY_TEXT"])
    ]
    for r in long_running
  ]
})

```

<h2>6. Error Log - Recent Errors / Warnings</h2>

```

{table(
  ["Time", "Priority", "Error Code", "Subsystem", "Message"],
  [
    [
      escape(r["LOGGED"]),
      escape(r["PRIO"]),
      escape(r["ERROR_CODE"]),
      escape(r["SUBSYSTEM"]),
      escape(r["MESSAGE"])
    ]
    for r in errors
  ]
})

```

<h2>7. Replication</h2>

```

"""

```

if replication is None:

```

    html += """
<div class="card">
Replication: <b>Not Configured / Not a Replica</b>
</div>
"""

```

elif "error" in replication:

```

    html += f"""
<div class="card">
Replication Check Error:

```

```
{escape(replication['error'])}  
</div>  
"""
```

```
else:
```

```
    repl_status = (  
        "GREEN"  
        if replication["io"] == "Yes"  
        and replication["sql"] == "Yes"  
        else "RED"  
    )
```

```
    html += table(  
        ["Metric", "Value"],  
        [  
            [  
                "Replica IO",  
                badge(replication["io"], repl_status)  
            ],  
  
            [  
                "Replica SQL",  
                badge(replication["sql"], repl_status)  
            ],  
  
            [  
                "Replication Lag",  
                str(replication["lag"])  
            ],  
  
            [  
                "Last IO Error",  
                escape(replication["last_io_error"])  
            ],  
  
            [  
                "Last SQL Error",  
                escape(replication["last_sql_error"])  
            ]  
        ]  
    )
```

```
html += "<h2>8. Binary Logs</h2>"
```

```
if binlogs:
```

```
    html += table(  
        ["Metric", "Value"],  
        [  
            ["Binary Log Files", binlogs["files"]],  
            ["Total Binlog Size",  
                f"{binlogs['size_gb']:.2f} GB"]  
        ]  
    )
```

else:

```
html += "<div class='card'>Binary logging is disabled or unavailable.</div>"
```

```
html += f"""
```

```
<h2>9. MySQL Status</h2>
```

```
{table(
    ["Metric", "Value"],
    [
        ["Threads Connected",
         mysqlstats.get("Threads_connected", "0")],

        ["Threads Running",
         mysqlstats.get("Threads_running", "0")],

        ["Threads Created",
         mysqlstats.get("Threads_created", "0")],

        ["Aborted Clients",
         mysqlstats.get("Aborted_clients", "0")],

        ["Aborted Connections",
         mysqlstats.get("Aborted_connects", "0")],

        ["Temporary Tables",
         mysqlstats.get("Created_tmp_tables", "0")],

        ["Disk Temporary Tables",
         mysqlstats.get("Created_tmp_disk_tables", "0")]
    ]
)}
```

```
<h2>10. DBA Action Items</h2>
```

```
<div class="card">
"""
```

```
actions = []
```

```
if health["disk_percent"] >= DISK_CRITICAL:
```

```
    actions.append(
        "<span class='critical'>CRITICAL: Disk usage above 90%</span>"
    )
```

```
elif health["disk_percent"] >= DISK_WARNING:
```

```
    actions.append(
        "<span class='warning'>WARNING: Disk usage above 80%</span>"
    )
```

```
if health["memory_percent"] >= MEMORY_CRITICAL:
```



```

        actions.append(
            "<span class='critical'>CRITICAL: Memory usage above 90%</span>"
        )

    elif health["memory_percent"] >= MEMORY_WARNING:

        actions.append(
            "<span class='warning'>WARNING: Memory usage above 80%</span>"
        )

    if connections["utilization"] >= CONNECTION_CRITICAL:

        actions.append(
            "<span class='critical'>CRITICAL: Connection utilization above 90%</span>"
        )

    elif connections["utilization"] >= CONNECTION_WARNING:

        actions.append(
            "<span class='warning'>WARNING: Connection utilization above 80%</span>"
        )

    if long_running:

        actions.append(
            f"<span class='warning'>WARNING: "
            f"{len(long_running)} long-running query(s) detected</span>"
        )

    if replication:

        if replication.get("io") == "No":

            actions.append(
                "<span class='critical'>CRITICAL: Replication IO thread is stopped</span>"
            )

        if replication.get("sql") == "No":

            actions.append(
                "<span class='critical'>CRITICAL: Replication SQL thread is stopped</span>"
            )

    if not actions:

        actions.append(
            "<span style='color:green;font-weight:bold'>"
            "No immediate DBA action required."
            "</span>"
        )

    for action in actions:

        html += f"<p>{action}</p>"

    html += ""

```

</div>

</body>

</html>

"""

```
filename = os.path.join(
    REPORT_DIR,
    f'mysql_dba_report_{now.strftime('%Y%m%d')}.html'
)
```

```
with open(filename, "w") as f:
    f.write(html)
```

```
conn.close()
```

```
print(f"Report created: {filename}")
```

```
# =====
```

```
# MAIN
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        generate_report()
```

```
    except Exception as e:
```

```
        print("ERROR:", e)
```

```
    raise
```

2. Create the MySQL monitoring user

Don't run the report as root.

For MySQL 8:

```
CREATE USER 'dba_report'@'localhost'
IDENTIFIED BY 'CHANGE_TO_STRONG_PASSWORD';
```

```
GRANT PROCESS,
    REPLICATION CLIENT,
    SELECT
ON *.* TO 'dba_report'@'localhost';
```

The sys schema can then be used for DBA-oriented monitoring because it summarizes Performance Schema information.

If the report server connects remotely:

```
CREATE USER 'dba_report'@'10.10.10.%'
IDENTIFIED BY 'CHANGE_TO_STRONG_PASSWORD';
```

```
GRANT PROCESS,
    REPLICATION CLIENT,
    SELECT
```

ON *.* TO ['dba_report'@'10.10.10.%'](#);

The sys schema can then be used for DBA-oriented monitoring because it summarizes Performance Schema information.

If the report server connects remotely:

3. Install Python requirements

On Linux:

```
sudo dnf install python3 python3-pip
```

Then :

```
pip3 install mysql-connector-python psutil
```

Test :

```
python3 /opt/mysql-dba-report/mysql_daily_report.py
```

You should get:

Report created:

```
/opt/mysql-dba-report/reports/mysql_dba_report_20260818.html
```

4. Schedule it every morning

For example, run at 7:30 AM:

```
Crontab -e
```

Add :

```
30 7 * * * /usr/bin/python3 /opt/mysql-dba-report/mysql_daily_report.py >> /opt/mysql-dba-report/report.log 2>&1
```

Now every morning:

07:30

↓

Python connects to MySQL

↓

Collects DBA metrics

↓

Creates HTML

↓

```
/opt/mysql-dba-report/reports/
```

↓

```
mysql_dba_report_YYYYMMDD.html
```